SYSTEM DESIGN MORNING REVISION

Use the ID tracker to choose five questions each morning. Cover the answer, speak for 60-90 seconds, then check the decision, trade-off, and failure. Short answers are prompts for recall, not scripts to memorize.

01 Requirements and Estimation

Q001 What do you clarify before drawing a system design?

Clarify users, core actions, what is out of scope, expected scale, latency, availability, durability, consistency, privacy, and cost constraints. Rank the requirements because they conflict. A design cannot optimize everything equally, and the most important requirement should explain the first major architecture decisions.

Q002 Why estimate traffic instead of saying the system must scale?

Traffic estimates convert vague scale into decisions. Peak reads, writes, object size, retention, and concurrency reveal whether one database is enough, whether media needs a CDN, and whether async processing is required. Use orders of magnitude and state assumptions; false precision is less useful than a testable capacity model.

Q003 How do you convert daily traffic into peak QPS?

Multiply active users by actions per user, divide by 86,400 seconds, then apply a peak factor based on the product. Separate reads from writes and account for retries and background work. Average QPS is a starting point; queues and saturation usually appear during concentrated peaks.

Q004 What is the difference between latency and throughput?

Latency is how long one operation takes. Throughput is how many operations complete per unit time. A system can have high throughput and bad latency because requests wait in queues. Always inspect percentiles and concurrency with throughput; one number does not describe user experience or remaining capacity.

Q005 Why is p99 important when average latency looks healthy?

The average can hide a slow minority caused by one shard, region, route, or dependency. p99 shows the value below which 99 percent of requests complete. Compare p50, p95, and p99 by cohort to find whether a tail problem affects all users or a specific path.

Q006 What is an SLO, and how should it influence design?

An SLO is a measurable target for a user-facing indicator, such as 99.9 percent successful feed loads under 500 ms over 30 days. It guides capacity, redundancy, alerting, and release decisions. Choose it from business impact and achievable performance, not because another company advertises more nines.

Q007 What are RTO and RPO?

RTO is the maximum acceptable time to restore service after failure. RPO is the maximum acceptable recent data loss, measured in time. They determine backup frequency, replication mode, standby capacity, and failover automation. A lower RTO or RPO normally costs more and introduces more coordination complexity.

Q008 How do you estimate storage growth?

Estimate new records per day multiplied by average bytes, retention, indexes, replicas, metadata, and safety margin. Calculate media separately from database rows because images and video dominate capacity and bandwidth. Include deletion policy and growth rate, then validate actual size from production samples.

Q009 Why estimate bandwidth and not only storage?

A system may store data comfortably but fail while transferring it. Multiply response or media size by peak request rate, include upload traffic and replication, and distinguish internal from internet egress. Bandwidth estimates often justify compression, object storage, a CDN, regional placement, and smaller API responses.

Q010 How much capacity headroom should a system keep?

Keep enough headroom for normal spikes, one-instance loss, deploys, and forecast error. The exact margin depends on scaling speed and dependency limits. Too little causes nonlinear queueing; too much wastes money. Measure saturation, rehearse failures, and revise the margin rather than using one universal percentage.

02 Scaling and Traffic Management

Q011 What is vertical scaling versus horizontal scaling?

Vertical scaling gives one machine more CPU, memory, or I/O. It is simple but has hardware limits and a larger failure domain. Horizontal scaling adds instances and can improve capacity and availability, but requires stateless request handling, load balancing, shared state, and control of pressure on downstream systems.

Q012 What makes an application service stateless?

Any healthy instance can handle the next request because required durable state lives in shared systems or travels with the request. Instance memory may hold disposable caches, but not the only session, upload, or workflow state. Statelessness makes rescheduling, rolling deploys, and horizontal scaling predictable.

Q013 What does a load balancer actually do?

It accepts traffic, selects a healthy backend, and may terminate TLS, route by host or path, enforce limits, and collect metrics. Its algorithm can be round-robin, least-connections, weighted, or locality-aware. It needs readiness signals and graceful draining so it does not send work to starting or terminating instances.

Q014 When is round-robin load balancing a poor choice?

Round-robin assumes requests and instances have similar cost. It performs poorly when requests vary greatly, connections are long-lived, or instances have unequal capacity. Least-connections, weighted routing, or latency-aware routing may fit better, but each can react badly to stale measurements and needs safeguards.

Q015 What is the difference between liveness and readiness?

Liveness answers whether the process should be restarted. Readiness answers whether it should receive traffic now. A service can be alive but unready during startup, draining, or dependency loss. Incorrect liveness checks can create restart loops; incorrect readiness checks can route users to an unusable instance.

Q016 Why can adding application instances make performance worse?

Every instance may open a database pool, issue concurrent queries, or retry a slow dependency. If that shared dependency is saturated, more callers increase contention and queueing. Scale the constrained tier or reduce work first, and cap total downstream concurrency rather than multiplying pressure blindly.

Q017 What signals should drive autoscaling?

Use the signal closest to the constraint: active requests, queue depth, request latency, CPU for CPU-bound work, or consumer lag for workers. CPU alone misses blocked requests and external bottlenecks. Scaling must also respect startup time, cooldowns, maximum capacity, and downstream headroom.

Q018 What is backpressure?

Backpressure makes producers slow down or receive rejection when consumers cannot keep up. It prevents unbounded queues, memory growth, and stale work. Implement it with bounded queues, concurrency limits, credits, or explicit rejection. The product must decide which work waits, degrades, or is dropped first.

Q019 What is load shedding?

Load shedding rejects or degrades lower-priority work before the entire system collapses. Examples include smaller feed pages, disabling recommendations, or returning 429 or 503. It should preserve critical operations, be measurable, and recover gradually. Shedding is a safety mechanism, not a substitute for capacity planning.

Q020 Why are sticky sessions usually a weak scaling solution?

They route a user repeatedly to one instance, which can temporarily support local session state. They create uneven load and fail when that instance disappears. Shared durable state or self-contained identity lets any healthy instance continue. Sticky routing may be a migration aid, not the final reliability model.

03 Databases and Query Design

Q021 How do you choose SQL versus NoSQL?

Start with access patterns, invariants, transaction boundaries, query flexibility, scale, and failure behavior. SQL fits relationships and constraints; key-value or wide-column stores fit known partitionable queries at high scale. Use multiple stores only when a measured need justifies synchronization, backup, security, and operational complexity.

Q022 What is an invariant, and why does it matter?

An invariant is a rule that must remain true, such as one like per user and post or a username being unique. The data model and transaction boundary should enforce it near the authoritative state. Application-only checks can race when concurrent requests observe the same old value.

Q023 What is database normalization?

Normalization separates facts to reduce duplication and update anomalies. It is valuable for authoritative transactional data. At read scale, selected denormalization may avoid expensive joins, but then every copy needs ownership, propagation, repair, and staleness rules. Normalize by default; denormalize for a proven access path.

Q024 What is an index, and what does it cost?

An index is an ordered data structure that helps the database find rows without scanning the full table. It speeds matching filters and sorts but consumes storage, memory, and write work. Design indexes from actual queries and inspect execution plans; indexing every column can reduce overall performance.

Q025 Why does column order matter in a composite index?

An index is ordered by its leading columns first. A query filtering author and ordering by creation time may benefit from `(author_id, created_at, id)`, while a query filtering only creation time may not. Match equality filters, ranges, and sort order to the dominant access pattern and verify the plan.

Q026 What is an N+1 query problem?

The application runs one query for a list and then one more query per item, such as checking like state for every post. Query count grows with page size and can exhaust connections. Fix it with a join, batch query, projection, or precomputed view, then measure rows and latency.

Q027 Why paginate large result sets?

Pagination bounds database work, response size, memory, and user wait time. Without it, one request grows with the full dataset and eventually dominates resources. Use a stable ordering and explicit limit. Cursor pagination is usually safer than deep offsets for frequently changing feeds.

Q028 Offset pagination versus cursor pagination?

Offset pagination is simple and supports page numbers, but deep offsets scan or discard more rows and concurrent inserts can shift results. Cursor pagination continues after a stable ordered key, making deep reads efficient and consistent. It requires opaque cursor management and does not naturally jump to arbitrary pages.

Q029 What is a database execution plan?

It shows how the database will scan indexes or tables, join rows, sort, and estimate cost. Compare estimated and actual rows, I/O, and timing. A slow query should be diagnosed with its real parameters and plan before adding hardware or indexes based only on intuition.

Q030 When should you add a read replica?

Add replicas after reducing unnecessary work, when stale-tolerant reads still exceed leader capacity or need regional locality. Keep writes and freshness-critical reads on the leader or apply read-your-writes routing. Monitor replication lag and stop routing when staleness exceeds the product contract.

04 Replication, Consistency, and Sharding

Q031 What problem does replication solve?

Replication copies data to improve availability, read capacity, locality, or disaster recovery. It does not automatically create a backup because deletion or corruption may replicate too. Replication introduces lag, failover, conflict, and promotion decisions that must be tested rather than assumed.

Q032 What is strong consistency?

After a successful write, subsequent reads observe that write according to the system's consistency model. It simplifies invariants but can add coordination latency or reduce availability during partitions. Use it where stale decisions are unsafe, such as ownership, authorization, uniqueness, or financial state.

Q033 Where is eventual consistency acceptable?

It fits derived or non-critical views where temporary staleness is understood: feeds, like counters, search indexes, analytics, and notifications. The authoritative write must remain correct, lag needs an SLO, and users may need read-your-writes. Eventual does not mean unbounded or unobservable inconsistency.

Q034 What does CAP theorem actually say?

When a network partition prevents nodes from communicating, a distributed data system must choose between always returning a response and preserving one consistent value. CAP does not say to choose two properties during normal operation. State the operation, partition behavior, and business consequence instead of using CAP as a slogan.

Q035 What is quorum reading and writing?

With N replicas, a write reaches W and a read consults R. If R plus W exceeds N, the read and write sets overlap, improving the chance of seeing recent data. Quorums still need versioning, repair, and conflict rules and do not magically provide every transactional guarantee.

Q036 What is replication lag?

It is the delay between a leader commit and a replica applying that change. Lag can make a successful update appear lost. Measure time and byte lag, route freshness-critical reads appropriately, and remove lagging replicas from service. Capacity, network, locks, and large transactions can all increase it.

Q037 What is sharding?

Sharding divides a dataset across independent owners so storage and traffic can exceed one node's limits. A shard key routes requests. It adds cross-shard query, transaction, rebalancing, hotspot, and operational complexity, so tune and scale a single database first when it still meets requirements.

Q038 How do you choose a shard key?

Choose from required queries and write distribution. The key should route common operations to one shard, have enough cardinality, and avoid known hot entities. Also plan secondary lookups and rebalancing. A key that distributes bytes evenly can still distribute request rate badly.

Q039 What is a hot partition?

One key or shard receives disproportionate traffic or data, so it saturates while fleet averages look healthy. Detect per-key and per-shard skew. Mitigate with better keys, salting, bucketing, caching, rate limits, or a special celebrity path, accepting extra read aggregation or routing logic.

Q040 How do you rebalance shards safely?

Use versioned routing, copy bounded ranges, validate counts and checksums, and handle writes during movement with a documented strategy. Shift reads gradually, then writes, and keep rollback until verification completes. Rebalancing must be idempotent and observable because partial movement is expected.

05 Caching and Content Delivery

Q041 What problem does a cache solve?

A cache keeps reusable data closer to callers, reducing latency and load on slower systems. It is useful only when repeated reads and acceptable freshness justify it. A cache introduces invalidation, eviction, stampede, memory, and consistency failures, so the source of truth must remain explicit.

Q042 What is cache-aside?

The application checks the cache, reads the database on a miss, then stores the result. Writes update the database and invalidate or update the cache. It is simple and demand-driven, but misses add latency and concurrent misses can stampede the source. Deletion and privacy changes need careful invalidation.

Q043 What is write-through caching?

Writes go through the cache layer, which updates the durable store before reporting success. Reads see a warm cache, but the cache becomes part of the write path and its failure affects availability. It does not remove the need for durability, ordering, retry, and consistency rules.

Q044 How do you choose a cache TTL?

Choose TTL from tolerated staleness, update frequency, rebuild cost, and memory pressure. Security or privacy-sensitive data may require explicit invalidation rather than waiting. Add random jitter so many keys do not expire together. Measure hit rate and stale-read impact instead of choosing one global value.

Q045 What is a cache stampede?

Many requests miss or observe one expired hot key and all rebuild it from the database. That can overload the source. Use single-flight locking, early refresh, stale-while-revalidate for safe data, TTL jitter, or prewarming. The lock itself must have timeout and failure handling.

Q046 What is negative caching?

It temporarily caches a not-found result to prevent repeated expensive lookups for missing keys. It helps against typos and abuse but can hide a newly created resource until expiry. Use a short TTL and never let a stale negative bypass current authorization or creation semantics.

Q047 What is cache penetration?

Requests repeatedly ask for keys that do not exist, bypassing a cache that stores only positive results and hitting the database. Negative caching, input validation, Bloom filters, and rate limiting can help. Confirm the request pattern is legitimate before adding probabilistic structures or blocking users.

Q048 What is a CDN?

A content delivery network caches objects at geographically distributed edge locations. It reduces user latency, origin bandwidth, and load for static or immutable content. It adds cache-control, purge, signed access, stale-content, and egress concerns. Use versioned immutable media URLs when possible.

Q049 How do you invalidate cached deleted content?

Mark deletion in the authority, prevent future authorization, publish a tombstone or purge request, and use versioned keys so old entries are not reused. For sensitive content, the serving path should check current permission or revoke signed access. TTL alone may be too slow.

Q050 What should never be used as a cache key or metric label carelessly?

Unbounded user-controlled values can explode memory or monitoring cardinality. Normalize routes and use bounded dimensions. Cache keys may include user identity when personalization demands it, but then sharing drops and invalidation grows. Never cache one user's authorized response under a public key.

06 Messaging and Distributed Workflows

Q051 When should work remain synchronous?

Keep work synchronous when the API response would be untruthful without it, such as authentication, validation, or committing the authoritative record. Move optional or slow side effects behind a durable boundary. Synchronous calls add immediate consistency but couple latency and availability to every dependency.

Q052 When should you use a queue?

Use a queue when work can happen later and one worker in a group should process each item, such as thumbnail generation. It smooths bursts and isolates request latency. It adds backlog, duplicate delivery, poison-message, ordering, retention, and operational concerns that need explicit limits and monitoring.

Q053 When should you use pub/sub?

Use pub/sub when several independent capabilities need the same fact, such as feed fanout, notifications, and analytics consuming `PostCreated`. Each subscriber gets its own progress and failures. The producer is decoupled, but contracts, retention, lag, replay, and ownership become operational responsibilities.

Q054 What delivery guarantee should you assume from a broker?

Assume at-least-once unless the complete end-to-end system proves something stronger. A worker can process a message and lose the acknowledgement, causing retry. Stable event IDs and idempotent consumers are therefore required. Broker exactly-once features do not cover every external database or side effect.

Q055 What is a transactional outbox?

The service writes the business change and an event record in the same local database transaction. A relay publishes pending events with retry. This prevents the gap where the database commits but the process crashes before publishing. Consumers still need deduplication because the relay may publish more than once.

Q056 What is an idempotent consumer?

It can receive the same event repeatedly without applying the logical effect more than once. Store the event ID with the local state change in one transaction or design the operation itself to be naturally idempotent. An in-memory seen set is insufficient because it disappears on restart.

Q057 What is a dead-letter queue?

It isolates messages that cannot succeed after bounded retries so they do not block healthy traffic forever. A dead-letter queue is not a trash can: preserve error context, ownership, alerts, replay tooling, and retention. Fix the cause before replaying or the same failure repeats.

Q058 What is a poison message?

A poison message deterministically fails a consumer because of invalid data, an unsupported schema, or a code defect. Repeated retries waste capacity and can block an ordered partition. Quarantine it after evidence-preserving attempts, repair the consumer or event, then replay idempotently.

Q059 What is a Saga?

A Saga coordinates a multi-step workflow using local transactions and messages. If a later step fails, compensating actions restore an acceptable business state. It avoids one global lock but exposes temporary states, retries, and complex recovery. Use a single database transaction when it can safely contain the workflow.

Q060 Orchestration versus choreography in a Saga?

Orchestration uses one coordinator to command steps and record state, making flow and recovery visible but centralizing logic. Choreography lets services react to events, reducing central control but making the overall path harder to understand. Choose based on workflow complexity, ownership, and debugging needs.

07 Reliability and Resilience

Q061 Why must every remote call have a timeout?

Without a timeout, callers can wait indefinitely, holding threads, memory, and connections until the service collapses. Set a deadline from the end-to-end latency budget, not an arbitrary large value. A timeout creates an unknown outcome for writes, so retries require idempotency or status lookup.

Q062 When are retries safe?

Retry transient failures only when the operation is idempotent or protected by an idempotency key. Use a small attempt limit, exponential backoff, jitter, and a total deadline. Do not retry validation errors or overload blindly. Multiple retry layers can multiply one failure into a traffic storm.

Q063 Why add jitter to exponential backoff?

Without jitter, many clients fail together and retry at the same exponentially spaced moments, creating synchronized spikes. Random delay spreads attempts across time. Jitter reduces coordinated load but does not make an unsafe write retry safe; idempotency and a retry budget are still required.

Q064 What is a circuit breaker?

It stops sending requests to a dependency that is repeatedly failing, fails fast during an open period, then probes recovery. This prevents resource exhaustion and shortens failure latency. It needs correct thresholds and a fallback; aggressive breakers can open during brief noise and hide partial recovery.

Q065 What is bulkhead isolation?

Bulkheads give workloads separate bounded resources so one failure cannot consume everything. Use distinct thread pools, connection limits, queues, or worker groups for critical and optional paths. Isolation can waste capacity and requires sizing, but it protects essential journeys from a noisy neighbor.

Q066 What is graceful degradation?

The system preserves its core promise while disabling or simplifying optional features. A feed might omit recommendations, serve cached results, or reduce page size. Degradation must be product-approved, observable, and safe. Never degrade authorization, data ownership, or other correctness rules silently.

Q067 What is a single point of failure?

It is one component whose loss stops the required service. Remove it with redundancy, independent failure domains, failover, and tested data recovery. Replicating application instances is insufficient if they share one database, region, secret store, network path, or operator dependency.

Q068 What is split brain?

Two nodes or regions both believe they are the authoritative writer, often after a partition or failed failover. Conflicting histories result. Prevent it with leases, consensus, quorum, and fencing. Recovery requires selecting authority and reconciling, not simply reconnecting both writers.

Q069 What is a health check allowed to verify?

Liveness should verify the process can make progress, not every dependency. Readiness may check whether the instance can safely serve its route, but deep checks can amplify outages. Measure dependency health separately and define degraded behavior so one slow service does not restart an otherwise healthy application.

Q070 How do you prevent a cascading failure?

Use deadlines, bounded concurrency, backpressure, load shedding, retry budgets, circuit breakers, and bulkheads. Watch for queues and dependency saturation before errors spike. Protect shared resources and critical paths. The goal is to contain failure and recover gradually, not merely retry until everything fails together.

08 APIs, Identity, and Abuse Protection

Q071 What makes an API backward compatible?

Existing clients continue working without coordinated deployment. Prefer additive fields, tolerant readers, stable meanings, and explicit deprecation. Do not repurpose fields or make optional data suddenly required. Test important consumers and keep a compatibility window before removing old behavior.

Q072 When should an API be versioned?

Version when behavior or representation must change incompatibly and coexistence cannot be handled additively. URI, header, and media-type versioning each work if governance is consistent. Versioning creates maintenance cost, so first try additive evolution, adapters, and deprecation rather than creating versions for every change.

Q073 What is an idempotency key at an API boundary?

It identifies one logical write across retries. The server stores the key, request fingerprint, status, and result for a retention period. Reusing the same key with a different payload should fail. It prevents duplicate creation when the original response is lost after the write commits.

Q074 How should access and refresh tokens differ?

Access tokens are short-lived credentials presented frequently to APIs. Refresh tokens are longer-lived, used only to obtain replacements, and need stronger storage, rotation, and revocation. A refresh token must not be accepted as an access token; include and validate token type, issuer, audience, and expiry.

Q075 How does refresh-token rotation detect theft?

Each refresh consumes the old token and creates the next token in one family. If an already-consumed token appears again, two parties may hold the chain, so revoke the family and require reauthentication. Store hashes, not usable plaintext tokens, and perform reuse detection atomically.

Q076 Authentication versus authorization?

Authentication proves who the caller is. Authorization decides whether that identity may perform this action on this resource. A valid token does not grant ownership of every post. Enforce resource-level authorization near the data and test forbidden cross-user access, not only missing credentials.

Q077 How do you rate-limit fairly?

Use separate policies by route cost and risk. Key authenticated traffic by principal and anonymous traffic by layered IP, device, and account signals. Token buckets allow controlled bursts. Monitor false positives, return Retry-After, and keep local emergency limits if the shared limiter fails.

Q078 Rate limiting versus concurrency limiting?

Rate limiting bounds arrivals over time. Concurrency limiting bounds work currently using a finite resource. A service may accept an average rate but still collapse when slow requests overlap. Use both when needed: rate controls abuse and demand; concurrency directly protects workers, connections, and dependencies.

Q079 How should secrets and signing keys be managed?

Store them in a dedicated secret manager, restrict access by workload identity, rotate with overlapping verification, audit use, and never commit them. Include a key ID so verifiers select the correct public key. Test emergency rotation and avoid one long-lived shared secret across every environment.

Q080 How do you protect a media upload endpoint?

Authenticate, authorize, enforce size and type limits, stream rather than buffer large files, use generated object keys, scan or quarantine content, and rate-limit expensive work. Store outside the application filesystem. Never trust the filename or MIME type alone, and remove orphaned uploads through reconciliation.

09 Feed, Media, Search, and Ranking

Q081 How should large media uploads work at scale?

The API authenticates and issues a short-lived pre-signed object-store upload. The client uploads bytes directly, then the backend verifies completion and records metadata. This removes large payloads from application servers. Use quarantine, checksum, size limits, lifecycle cleanup, and explicit states for incomplete uploads.

Q082 Why use object storage instead of application disk?

Object storage is durable, shared across replicas, scalable, and integrates with CDN delivery and lifecycle rules. Local disk disappears with instance replacement and creates pod-specific results. Keep metadata and ownership in the database, store immutable object keys, and treat object operations as a separate workflow.

Q083 How do you process thumbnails and media variants?

After upload verification, publish a durable event to idempotent workers. Workers validate content, generate named variants, store immutable objects, and update processing status. The UI can show a pending state. Bound retries, quarantine failures, and ensure repeated events do not create uncontrolled duplicate objects.

Q084 Fan-out on write versus fan-out on read?

Fan-out on write copies a new post into follower timelines, giving fast reads but high write amplification for popular creators. Fan-out on read gathers followed content at request time, lowering write cost but increasing read latency. Large social systems often push normal creators and pull celebrities, then merge candidates.

Q085 What is a celebrity problem in feed generation?

One creator has millions of followers, so synchronous or immediate fan-out overwhelms queues and storage. Detect high-fan-out authors and use a pull path, delayed batches, or priority tiers. Protect ordinary users' work and reconcile incomplete fan-out. A single strategy rarely fits both normal and celebrity accounts.

Q086 What should a feed cursor contain?

Use an opaque encoding of the stable ordering boundary, commonly `(created_at, id)`, plus version or filter context if needed. The next query asks for rows strictly after that tuple. A unique tie-breaker prevents duplicate or skipped items when timestamps are equal. Sign cursors if tampering matters.

Q087 How do you build text search?

Keep transactional data authoritative and asynchronously build an inverted index with analyzers for captions, hashtags, and usernames. Query returns ranked IDs; the serving layer applies current privacy and deletion rules before hydration. Use versioned events, tombstones, lag monitoring, reconciliation, and full rebuild capability.

Q088 Why can search be near-real-time but deletion still need stronger handling?

Users may accept a short delay before new content appears, but deleted or private content remaining visible is a safety failure. Publish tombstones, reject older event versions, purge cached results, and enforce authorization at read time. Freshness requirements differ by operation, even within one index.

Q089 How do you rank a personalized feed?

Generate a bounded candidate set, apply privacy and safety filters, score with available features, add diversity rules, and hydrate the top page. Keep ranking within a deadline and provide a recency fallback. Evaluate offline, canary changes, and measure latency, safety, freshness, and user value together.

Q090 How do you handle cold start in recommendations?

Use explicit onboarding interests, followed accounts, recency, locale-safe popularity, and controlled exploration. New content can use creator and content features until interaction history develops. Do not pretend personalization exists when data is absent. Keep a transparent fallback and measure whether exploration harms quality or safety.

10 Observability and Performance

Q091 What are the three pillars of observability?

Metrics summarize rates and trends, logs record detailed events, and traces show work across components. They should share bounded service, route, version, and correlation context. The pillars are tools, not the goal: define user journeys and questions first, then collect signals that support decisions.

Q092 What are RED and USE methods?

RED monitors request Rate, Errors, and Duration for services. USE monitors Utilization, Saturation, and Errors for resources. RED shows user-facing symptoms; USE helps locate constrained CPU, memory, disks, pools, or queues. Combine them with traces and business outcomes instead of choosing only one framework.

Q093 Why must metric labels be bounded?

Every unique label combination creates a time series. User IDs, raw URLs, or request IDs can create millions of series, overwhelming storage and queries. Use normalized routes and controlled enums in metrics. Put high-cardinality diagnostic identifiers in access-controlled logs or sampled traces instead.

Q094 What is distributed tracing useful for?

A trace follows one request or event through services and shows where time and errors occur. It helps separate gateway wait, application work, database calls, and downstream latency. Sampling controls cost, but retain errors and slow traces preferentially. Traces should complement population-wide metrics, not replace them.

Q095 How do you investigate a peak-hour latency spike?

Compare baseline and peak request rate, errors, p50, p95, p99, concurrency, queues, pools, database health, and downstream latency. Break down by route, instance, region, and version. Trace slow requests to the first major wait. Mitigate the constrained tier, then reproduce the load and verify recovery.

Q096 What does high latency with low CPU suggest?

Requests may be waiting for database connections, locks, disk, remote calls, executor queues, or rate limits. CPU measures computation, not all saturation. Check active and waiting counts for every bounded resource and inspect slow traces. Adding application instances may worsen a blocked shared dependency.

Q097 What is queueing delay?

It is time spent waiting before actual service begins because a finite worker, connection, or device is busy. Near capacity, small traffic increases can cause nonlinear waiting. Measure queue depth, wait time, utilization, and service time. Larger queues often hide errors by converting them into worse latency.

Q098 What is a connection-pool saturation failure?

All database connections are busy, so request threads wait even when application CPU is low. Causes include slow queries, long transactions, locks, leaks, and excess concurrency. Find the cause before enlarging the pool; more connections can overload the database and increase contention.

Q099 How do you detect a retry storm?

Dependency failures rise, client attempts exceed original requests, queues grow, and recovery slows because retries consume remaining capacity. Trace attempt counts and compare incoming demand with downstream calls. Stop or limit retries, open breakers, shed work, and restore one layer at a time.

Q100 What should an incident dashboard show first?

Show critical journey success, latency percentiles, traffic, and freshness, then resource saturation and dependency health. Add deployment markers and break down by version, region, route, and instance. A dashboard should support a decision: scope impact, identify the first change, and choose a safe mitigation.

11 Deployment, Migration, and Recovery

Q101 What is a rolling deployment?

Instances are replaced gradually while old and new versions coexist. It limits capacity loss but requires backward-compatible APIs, schemas, and events. Readiness and graceful draining prevent traffic from reaching unready or terminating instances. Monitor version-separated metrics and stop rollout when guardrails fail.

Q102 What is a canary deployment?

A small production population receives the new version first. Compare its latency, errors, saturation, and business outcomes with the stable version under similar traffic. Canarying limits blast radius and produces causal evidence. It needs representative traffic, automatic guardrails, and a fast routing or rollback mechanism.

Q103 Blue-green versus rolling deployment?

Blue-green prepares a complete parallel environment and switches traffic, making rollback fast but doubling temporary capacity and requiring data compatibility. Rolling replacement uses less extra capacity but mixes versions longer. Choose from cost, deployment speed, state compatibility, and failure isolation needs.

Q104 What should you check before rolling back?

Confirm the previous binary is compatible with current schema, data, events, and configuration. A destructive migration may make rollback unsafe. Preserve evidence, pause rollout, and choose rollback, feature disablement, traffic isolation, or focused roll-forward based on the safest reversible action.

Q105 How do you change a schema without downtime?

Use expand-migrate-contract. Add a compatible nullable field or new table, deploy code that handles both shapes, backfill in small restartable batches, verify, then add constraints and later remove the old shape. Keep expand and contract in separate releases so mixed versions remain safe.

Q106 How do you run a safe backfill?

Process bounded stable key ranges, checkpoint progress, make updates idempotent, and throttle based on production latency and replication lag. Monitor locks, I/O, errors, and row counts. Pause safely, resume without duplication, validate coverage, and keep the old read path until correctness is proven.

Q107 What belongs in a disaster recovery plan?

Per-journey RTO and RPO, dependency inventory, backup and restore, replication, standby capacity, routing, write fencing, access credentials, communication, and reconciliation. Test the complete failover at realistic load. A document or unused replica is not proof that recovery will work.

Q108 Active-passive versus active-active regions?

Active-passive has one write authority and simpler conflicts but slower failover and standby cost. Active-active can lower regional latency and improve availability but requires global invariants, conflict resolution, fencing, and harder operations. Use active-active writes only when business needs justify that complexity.

Q109 Why are backups required when replicas exist?

Replication can copy accidental deletion, corruption, or malicious changes to every replica. A backup provides an independent recovery point. Define retention, encryption, access, and restore objectives, then test restores regularly. An untested backup is only a hope, not recovery evidence.

Q110 How do you verify a regional failover?

Fence the old writer, measure replica lag, shift safe reads first, promote one authority, ramp traffic gradually, and confirm journey SLOs at destination capacity. Reconcile the known data gap and test failback separately. Control-plane success alone is insufficient; verify real user operations and durable data.

12 Architecture and Senior Trade-offs

Q111 When should a monolith remain a monolith?

Keep it when one deployment and database meet scale, reliability, and team needs. A modular monolith offers strong transactions and simple operations. Microservices are justified by independent scaling, isolation, ownership, or release needs. Splitting too early converts local calls into distributed failure without business benefit.

Q112 How do you choose a microservice boundary?

Choose a cohesive business capability with clear ownership, data, and language, not one table or controller. Minimize synchronous chatty calls and cross-service transactions. A good boundary can evolve and deploy independently. Start with modules and extract only where the operational benefit is measurable.

Q113 Why should one service own its data?

Ownership protects invariants and allows internal schema evolution. Other services use APIs or events rather than writing its tables directly. Shared database writes couple deployments and create unclear authority. Read projections may be copied, but one owner decides the authoritative state during disagreement.

Q114 What is a distributed monolith?

Services are deployed separately but must change together, call each other synchronously in long chains, share data, and fail as one system. It has network and operational cost without independence. Repair boundaries, contracts, ownership, and async flows before adding more services.

Q115 What is the strangler migration pattern?

Place a stable facade around a legacy system and route one bounded capability to a new implementation. Migrate traffic gradually, compare results, and preserve rollback while both coexist. Repeat until the old path can be removed. Start with a low-coupling capability, not the hardest core transaction.

Q116 Build versus buy: what should guide the choice?

Compare strategic differentiation, requirements, integration fit, security, reliability, cost over time, operational skill, vendor lock-in, and exit plan. Buy commodity capability when it safely meets needs; build what creates product advantage or requires unique control. Include migration and on-call cost, not only license price.

Q117 How do you present trade-offs in an interview?

Name the requirement, chosen option, benefit, cost, new failure mode, and alternative. Avoid calling a technology best practice without context. State what you would build first and what scale signal would trigger evolution. Clear sequencing demonstrates judgment better than listing every possible component.

Q118 How do you design for multi-tenancy?

Choose tenant isolation for data, compute, limits, encryption, and operations. Shared tables are efficient but need tenant keys and strict authorization; separate databases improve isolation but increase cost and fleet management. Prevent noisy neighbors with quotas and measure usage per tenant without unbounded metrics.

Q119 How do you handle configuration safely across environments?

Keep configuration external to the binary, validated, versioned, and least-privileged. Separate secrets from ordinary settings. Use safe defaults, staged rollout, audit history, and rollback. A configuration change can be as dangerous as code, so observe it with deployment markers and compatibility checks.

Q120 What is a strong closing summary for a system-design interview?

Restate the prioritized requirements, main request and data flow, source of truth, two critical trade-offs, top failures and mitigations, and the next scaling step. Mention what you deliberately left out. A concise close proves the design is coherent and gives the interviewer clear areas for follow-up.

13 Advanced Distributed-System Tools

Q121 What is service discovery?

It maps a logical service name to healthy network endpoints as instances start, stop, and move. Discovery may be client-side or handled by a proxy or platform service. It needs health integration, caching, and stale-endpoint handling. Hard-coded addresses break dynamic scaling and recovery.

Q122 What belongs in an API gateway?

A gateway can terminate TLS, authenticate, rate-limit, route, transform protocols, and collect edge metrics. Keep business rules in owning services so the gateway does not become a giant central application. Make it redundant, bound request sizes and timeouts, and preserve trace context.

Q123 What is consistent hashing?

It maps keys and nodes onto a ring so adding or removing a node moves only part of the keyspace. Virtual nodes improve balance. It is useful for distributed caches and partition routing, but replication, hot keys, membership changes, and failure recovery still require explicit design.

Q124 When is a distributed lock appropriate?

Use it only when one actor must perform a short exclusive action and the resource cannot enforce the rule itself. Locks need leases, fencing tokens, timeouts, and failure recovery. Prefer database constraints, idempotency, partition ownership, or single-writer design when they solve the problem more safely.

Q125 What problem does consensus solve?

Consensus lets nodes agree on one ordered decision despite failures, enabling leader election and replicated metadata. Algorithms such as Raft trade latency and availability for agreement. Do not implement consensus casually; use a proven system and keep the coordinated state small because quorum work is expensive.

Q126 What is leader election?

Nodes choose one active coordinator for work that must have a single owner. The leader holds a time-bounded lease and uses fencing so an expired leader cannot continue writing. Election improves failover but creates transition periods; clients must tolerate brief unavailability or retry safely.

Q127 What is CQRS?

Command Query Responsibility Segregation separates the write model from one or more read models. It helps when reads need very different shapes or scale, but introduces asynchronous propagation, duplicated data, repair, and more code paths. Do not use it when ordinary queries and one model meet the requirements.

Q128 What is event sourcing?

Event sourcing stores an append-only sequence of domain facts and derives current state by replay or snapshots. It provides history and temporal reconstruction but complicates schema evolution, privacy deletion, debugging, and projections. Use it when the event history is a core requirement, not merely for audit logging.

Q129 What is a Bloom filter?

A Bloom filter is a compact probabilistic set that can say an item is definitely absent or possibly present. It reduces expensive missing-key lookups with no false negatives but allows false positives. Size it from expected items and error rate, and rebuild it when membership changes significantly.

Q130 How do data residency and privacy affect architecture?

Classify data, define permitted regions and purposes, minimize collection, encrypt it, enforce least privilege, and audit access. Residency can constrain replication, backups, analytics, and support operations. Separate identity from analytical data where possible and define deletion across caches, indexes, replicas, and backups.

Q131 How should feature flags be used safely?

Flags decouple behavior release from binary deployment and provide fast containment. Define an owner, default, scope, expiry date, and tested off path. Avoid deeply nested permanent flags because they multiply states. Sensitive authorization rules should not depend on an unreviewed remote flag.

Q132 How do you control cost without weakening reliability blindly?

Attribute cost by service and journey, measure utilization and unit cost, then remove waste before reducing safety margin. Use storage lifecycle rules, right-sizing, autoscaling, caching, batching, and commitment discounts where demand is stable. Protect SLOs and recovery capacity, and verify savings with production evidence.